Parsing Documents at Scale and Extracting Knowledge

System design discussion, November 12, 2026. This write-up captures the full discussion in the order it happened, from ingestion through indexing, retrieval, evaluation, and tooling. It also records the gotchas raised along the way and the items pinned for a later deep dive.

Scope note: the discussion opened with a question about building a knowledge graph or a knowledge base, and the framing settled on a knowledge base, not really a knowledge graph. The design below still uses a graph store for entities and relations, because that came up when indexing was discussed. Both framings are recorded honestly: the product is a knowledge base built from parsed documents, with a graph store as one of its indexes, not a graph-first system.

1. The overall pipeline

The pipeline has six stages.

1. Ingest documents into durable object storage.
2. Queue documents for parallel parsing.
3. Run layout-aware extraction to get text, tables, and figures.
4. Chunk with overlap and embed for retrieval.
5. Store embeddings in a vector index.
6. Run entity and relation extraction into a graph store with provenance, and keep the raw chunks linked so every fact is traceable.

The pipeline is event driven. A document arriving or changing in the source system produces an event, the event drives parsing, parsing drives chunking and embedding, and embedding drives the indexes. Nothing in the pipeline rescans the whole corpus to find work. Each stage only acts on the documents the previous stage hands it.

Provenance is a first-class requirement, not an afterthought. Every chunk carries the document it came from, the version of that document, the page numbers, and where the original bytes live. Every entity and relation in the graph store links back to the chunk it was extracted from, and every chunk links back to the original document version. An answer is only as good as its citation, and the citation has to survive document updates.

2. Ingestion

Ingestion is split into a control plane and a data plane.

The control plane tracks what exists, what changed, and what has been processed. The data plane moves bytes and runs the work. Keeping them separate means the tracking state survives worker failures, and the workers can scale independently of the tracking store.

The flow:

- Files land in versioned storage.
- An event is emitted for each new or changed file.
- A fleet of workers pulls from the queue with backpressure, so a burst of documents does not overwhelm the parsing stage.
- Each worker runs a virus scan, then format detection, then a layout model that pulls text in reading order while preserving tables, headers, and page numbers as metadata.

- The original bytes are kept untouched. Nothing in parsing ever mutates the source file.
- A manifest is written with the content hash, the source, and the ingestion time.
- Every job is idempotent, so a retry does not double process a document.

Idempotency matters because queues redeliver. A worker can crash after writing output but before acknowledging the message, and the queue will hand the same message to another worker. The manifest, keyed by document and version, is what makes the second attempt a no-op instead of a duplicate.

2.1 The enterprise source and its API rate limit

The documents live in an enterprise SaaS system with an API rate limit. The design has to honor that limit, and it has to detect changes in the source without hammering the API or polling S3.

The answer is a dedicated fetcher service that sits outside the rate-limited system. It does three things:

1. It walks the source API on a schedule, using a token bucket to stay under the rate limit.
2. It detects changes by comparing metadata, not by downloading every document.
3. It writes only new or changed documents into S3, and emits one event per write.

Polling S3 constantly to find changes is the wrong approach. The fetcher knows what changed because it compared hashes at the source, and S3 events tell the rest of the pipeline what arrived. The bucket is storage, not a change log.

2.2 The fetcher loop

The fetcher keeps a cursor table with one row per document. On a schedule, it walks the source API, requests metadata only, compares the hash it sees against the hash it last stored, and when something changed it streams the bytes directly into S3. The S3 write triggers an event to SQS, so the parser picks it up. The fetcher also writes the manifest to Dynamo or Postgres, so there is an audit trail and any version can be replayed without rescanning the whole bucket.

The token bucket is what honors the rate limit. Tokens refill at the allowed rate, and each API call spends a token. When the bucket is empty, the fetcher waits. Bursts are absorbed by the bucket depth, and the long-run rate never exceeds what the source allows.

2.3 Cursor table schema

The cursor table is the single source of truth for what changed, when, and where it lives.

Column	Type	Notes
document_id	string	Primary key. Stable identifier from the source system.
source_url	string	Where the document lives in the source system.
last_seen_hash	string	Hash or etag from the source at last check. The change signal.
last_modified	timestamp	Last modified timestamp reported by the source.
s3_key	string	S3 key where this version was stored.
content_hash	string	Hash of the bytes actually written to S3.
fetched_at	timestamp	When this version was fetched.
status	string	pending, succeeded, or failed.
retry_count	integer	Number of fetch attempts for the current version.

Two hashes are kept on purpose. `last_seen_hash` is what the source reported, and `content_hash` is what was actually written. If they disagree, something went wrong between the source and the bucket, and the manifest is what catches it.

2.4 Storage layout

S3 has versioning enabled, so each write is immutable. The object key is the document ID plus a version timestamp, which makes every version addressable and keeps the history intact for

replay. The S3 event carries the key to the queue, so the ingestion worker gets both the raw bytes and the metadata in one go. Because the queue message is tied to a specific versioned key, each version is processed once.

3. Parsing

Once a worker has the file, it runs through a layout model that understands reading order. The goals:

- Paragraphs stay intact.
- Columns are not merged into each other.
- Text is extracted with its page number and bounding box.
- Tables are pulled into a structured format with headers preserved.
- Figures are captured as references, not dropped.

Reading order is the hard part. A naive text extractor reads a two-column PDF top to bottom and interleaves the columns. A layout model segments the page into blocks, orders them the way a reader would read them, and only then emits text. Page numbers and bounding boxes are kept as metadata on every block, because the citation at the end of the pipeline has to point at a real place in the original document.

Tables get special handling. A table flattened into running text loses its structure, and a question about a value in a row becomes unanswerable. Preserving headers and cell positions keeps the table queryable.

4. Chunking

Chunking turns parsed text into retrieval units. The strategy discussed is layout-aware chunking with overlap.

- Respect headings and page breaks, so chunks follow the document's own structure.
- Cut into overlapping chunks with about 20 percent overlap, so a fact near a boundary is not lost.
- Attach location metadata to every chunk: document ID, version, page numbers, chunk index.

The overlap is a safety net. Layout awareness gives semantic coherence, because a chunk follows a section rather than cutting mid-sentence. Overlap covers the case where the meaning still straddles a boundary. A fact that sits at the end of one chunk also appears at the start of the next, so retrieval finds it either way.

Each chunk is self-describing. It carries enough metadata to be traced back to the exact spot in the original PDF without any other lookup.

5. Embeddings

5.1 What an embedding is

An embedding is a numeric fingerprint for a chunk of text. The embedding model reads the chunk and outputs a vector, a list of several hundred numbers that represents its meaning. Chunks about similar topics end up with similar vectors, so meaning becomes distance: close vectors mean similar meaning.

Each overlapping chunk gets its own vector, and the vector is stored next to the chunk ID, the document version, the page, and the S3 location. When a question comes in later, the question

is embedded the same way, and the system compares the question vector against the stored chunk vectors to find the closest ones. Retrieval becomes a nearest-neighbor search over meaning, not a keyword match over words.

5.2 Why not just feed chunks to an LLM

The discussion raised the question directly: LLMs keep getting better, so why not skip embeddings, do plain text search, and feed the chunks to an LLM to do the work.

The answer is scale, cost, speed, and provenance.

- Feeding raw chunks to a large model works for a handful of documents. At the scale of a full enterprise corpus, it is slow and expensive, because the model has to read everything to answer anything.
- Embeddings give a fast first pass. They pull the most relevant chunks cheaply, and only those few go to the model, with their source links attached.
- Pure text search misses meaning. It matches words, not ideas, so a question phrased differently from the document finds nothing.
- Pure LLM over everything misses scale. It has no index, so every question costs a full scan.

The design keeps both pieces. Embeddings do the fast, cheap retrieval. The LLM does the expensive, careful answer generation, over a small set of cited chunks. That pairing is what makes the system affordable and traceable at the same time.

6. Indexing

6.1 What the vector store holds

Indexing means taking each chunk's vector and putting it into a vector database where it can be searched quickly by similarity. The store holds more than the vector. For each entry it keeps:

- The vector itself.
- The chunk text.
- The chunk ID.
- The document version.
- The page number.
- The S3 key pointing at the original document.

The metadata travels with the vector, so when a similarity search returns a match, the result carries the meaning, the words, and the source in one record. There is no second lookup to find where a chunk came from. That is what lets retrieval stay fast and still cite the original source.

LanceDB was discussed as an example of this kind of store. The principle is the same across vector databases: vector plus payload, searched by similarity.

6.2 How vector similarity works

Similarity is distance between vectors. Two chunks with similar meaning have vectors that sit close together, so the distance between them is small. The common measures:

- Cosine similarity, which looks at the angle between two vectors. It is the usual default for text embeddings, because it ignores magnitude and compares direction, which is where the meaning lives.

- Euclidean distance, the straight-line distance between two points in vector space.

At scale, the index does not check every vector. It uses approximate nearest neighbor (ANN) search:

- HNSW (Hierarchical Navigable Small World) builds a navigable graph over the vectors, so a search hops from node to node toward the closest matches instead of scanning everything.
- IVF (Inverted File Index) groups vectors into clusters and only searches the clusters most likely to contain matches.

Both trade a small amount of exactness for a large amount of speed. That tradeoff is what makes retrieval fast over millions of chunks.

6.3 What else indexing has to cover

Storing vectors is only the start. The rest of indexing:

- The ANN structure (HNSW or IVF) has to be built and maintained, so lookups stay fast as the index grows.
- Updates are incremental. New chunks arrive through the queue, and the index absorbs them without a full rebuild.
- Embeddings are versioned alongside the documents. When a document changes, its chunks are re-embedded and the old vectors are replaced, without losing the provenance of the old version.
- Each vector links to the graph store, so a retrieved chunk can also surface its entities and relations.
- Monitoring tracks freshness and recall, so the team knows the index reflects the latest documents and is still returning the right chunks.

6.4 What indexing is for

The vector index is built for question answering and search retrieval. A question becomes a vector, the index finds the closest chunk vectors, and the system returns the actual text with its source to feed to the model. The loop is: embed the question, retrieve by similarity, answer from the retrieved chunks, cite the source.

7. The change flow, end to end

When a document changes in the SaaS source, the whole pipeline reacts. Step by step:

1. The fetcher sees the new hash or timestamp on its next rate-limited check of the source API.
2. The fetcher streams the new bytes into S3 with a fresh key that includes the version timestamp. Versioning keeps the old version in storage.
3. The S3 write fires an event into SQS.
4. A parser worker picks up the message, reads the new bytes and the manifest entry, and runs layout-aware extraction again.
5. The parser produces updated overlapping chunks for the new version.
6. The new chunks get new embeddings.
7. The vector index upserts the new vectors, replacing the old vectors for that version. The old version stays in storage for history and replay.
8. The graph store updates entities and relations, with links back to the new chunk locations.
9. The flow stays idempotent throughout, so a retry or a redelivered queue message does not

duplicate anything.

10. Monitoring marks the index as fresh for that document version.

Nothing in this flow rescans the corpus. The change propagates from the source to the index through events, and each stage only touches the document that changed.

8. Retrieval and the Q and A layer

8.1 Semantic search versus text search

Semantic search is what the vector index does. It finds chunks by meaning, using vectors. Text search is the older style: it matches exact words or word stems. Text search is good for names, IDs, and phrases that have to appear exactly. Semantic search is good for ideas phrased differently from the document.

Neither alone is enough. Semantic search can miss an exact identifier. Text search can miss a paraphrase. The design runs both and blends the results.

8.2 The end-to-end query flow

When a user types a query:

1. The query is sent through the embedding model to get a vector.
2. In parallel, a keyword search runs against the chunk text for exact terms.
3. The query vector goes to the ANN index (HNSW or IVF) and pulls back the top chunks by meaning, each with a vector score, chunk ID, text, page, document version, and S3 location.
4. The keyword search pulls its own top chunks with a text relevance score.
5. Both score sets are normalized, because vector scores and text scores live on different scales.
6. The normalized scores are blended with a chosen weight, meaning weighted a bit higher for semantic relevance, text kept strong for identifiers.
7. The two lists are merged and deduplicated. A chunk found by both paths appears once, with the stronger combined score.
8. The metadata stays attached to every candidate through the merge.
9. The blended set becomes the context window. The chunks and the original question go to the model.
10. The model generates an answer that cites the specific chunks it used.
11. Behind the scenes, the system logs which chunks came from which path, so evaluation can tell whether one path is missing things the other finds.

The log of retrieval paths is what makes the blend tunable. If the keyword path consistently finds chunks the vector path misses, the weights or the chunking need work. Without that log, the blend is a guess.

8.3 Where LLMs are used

LLMs are used in three places only. The rest of the pipeline stays deterministic.

1. Parsing: a layout-aware model turns the PDF into clean text, tables, and structure.
2. Embeddings: a smaller encoder model turns each chunk and each query into vectors.
3. Answer generation: a larger generative model takes the blended top chunks with their

sources and writes the answer, citing where each fact came from.

Everything else, fetching, queuing, chunking, indexing, scoring, blending, logging, is deterministic code. That keeps the system fast, cheap, and debuggable. When something goes wrong, the deterministic stages can be replayed exactly, and only the three model stages need separate evaluation.

9. Evaluation at every stage

Evaluation is measured at each stage, not just at the end. A failure at ingestion looks different from a failure at retrieval, and the metrics have to tell them apart.

9.1 Ingestion

- Fetch success rate: successful fetches divided by total documents checked.
- Hash detection: whether the hash comparison caught the real change.
- Manifest match: whether the S3 version matches the manifest entry for that document and version.
- Alert if the fetch success rate drops below 99 percent.

A missed change at ingestion is silent downstream. Nothing later in the pipeline can recover a document that was never fetched, so ingestion metrics are the first line of defense.

9.2 Parsing

- Golden set: keep ten complex PDFs with tables and columns, with human-checked baselines for their extracted text and table structure. Run the parser against this set on every deploy.
- Text overlap: compare extracted text against the baseline.
- Table cell accuracy: compare extracted tables cell by cell against the baseline.
- Metadata presence: check that page numbers and bounding boxes are present on the extracted blocks.
- Human spot checks: sample parsed output for review.

Parsing drift is gradual. A layout model update or a new document format degrades extraction a little at a time, and the golden set is what catches it before users notice.

9.3 Chunking and embeddings

- Sentence boundary preservation: check that overlapping windows do not cut sentences in half.
- Overlap coverage: verify that the overlap actually carries content across boundaries.
- Near-duplicate similarity: duplicate chunks should have cosine similarity above the chosen threshold.
- Embedding stability: the same chunk embedded twice should produce near-identical vectors.

If embeddings drift, retrieval degrades everywhere at once. Stability checks catch model or preprocessing changes before they reach the index.

9.4 Retrieval

- Labeled question set: maintain questions where the correct answer, document version, and page are known.
- Recall at 5 and at 10, computed separately for vector search, keyword search, and the hybrid blend.
- Mean reciprocal rank: how high the first correct chunk ranks.
- Path comparison: compare what the vector path finds against what the keyword path finds, using the retrieval path log.

Recall at k answers the question "did the right chunk make it into the top k at all." Mean reciprocal rank answers "how far down did the user have to look." Both matter, because a correct chunk at rank 9 is much less useful than one at rank 1.

9.5 The answer layer

- Faithfulness: the answer uses only the retrieved chunks, nothing invented.
- Citation accuracy: every citation points to the correct document version in S3.
- Claim-to-chunk mapping: automatically verify that every sentence in the answer maps to a cited chunk.
- Human sample: review 5 percent of answers weekly, rated for faithfulness on a 1 to 5 scale.
- Latency and end-to-end accuracy: track how long answers take and how often they are right.

Faithfulness is the metric that matters most to users. A fast answer that cites the wrong version is worse than a slow answer that cites the right one.

10. Tools and technologies

The stack mixes managed AWS services with open source components.

| Stage | Technology | | --- | --- | | Change events | EventBridge | | Raw storage | S3 with versioning | | Work queue | SQS | | Fetcher | ECS or Lambda, with a token bucket for rate limiting | | Parsing | LayoutLM or Detectron2 in containers on ECS | | Manifest | DynamoDB | | Embeddings | Open embedding model served through SageMaker endpoints | | Vector search | OpenSearch Serverless with HNSW index | | Graph store | Neptune, with chunks linked back to S3 | | Answer generation | Model in Bedrock | | Evaluation runs | Step Functions, scheduled | | Metrics and dashboards | CloudWatch custom metrics | | Logs | CloudWatch Logs, structured | | Citation checks | Neptune queries verifying cited chunks exist in S3 at the right version |

10.1 Evaluation and monitoring in practice

Golden datasets live in S3. Step Functions runs the evaluation jobs on a schedule, and the scores for parsing, retrieval, and faithfulness are written to CloudWatch as custom metrics, with a dashboard over them. Structured logs from each stage go to CloudWatch Logs. Alarms cover ingestion success, parsing drift, embedding latency, and recall drops. OpenSearch slow query logs catch retrieval degradation. A Neptune query check verifies that every cited chunk still exists in S3 at the right version.

The same telemetry serves two purposes. Offline evaluation runs tell the team whether quality is improving or regressing. Live operational signals tell the team whether the system is healthy right now. Both feed the same dashboard, so a quality regression and an outage are visible in one place.

11. Gotchas

These are the pitfalls raised in the discussion, plus standard-practice additions that follow

directly from the design. Items marked with (standard practice) were not named in the discussion itself but are direct consequences of the architecture described.

- Parsing drift on PDFs. Layout models and document formats change. Extraction quality degrades gradually, and without a golden set nobody notices until answers go wrong.
- Chunk boundaries splitting meaning. Even with overlap, a long fact can straddle a boundary in a way that neither chunk carries fully. Overlap reduces the risk, it does not remove it.
- Stale embeddings. When a document changes and re-embedding lags, the index serves the old meaning. The freshness monitor has to cover the gap between a document change and its re-embedding.
- Duplicate entities. The same entity extracted from different chunks, or spelled differently across documents, becomes two nodes in the graph. Without deduplication and canonicalization, the graph fragments.
- Graph bloat without governance. Every extraction adds nodes and edges. Without pruning, schema discipline, and ownership rules, the graph grows faster than its usefulness.
- Idempotency failures. A queue redelivery or a worker crash mid-write can duplicate chunks, vectors, or entities. Every stage needs a deterministic key, usually document ID plus version plus chunk index, so a retry is a no-op.
- Rate-limit violations. A fetcher bug or a misconfigured token bucket can exceed the source API limit and get the integration throttled or blocked. The token bucket is a safety control, not just a throttle.
- Version skew between index and source. The index can hold vectors for a document version the source has already replaced. Any query answered from a stale version is a wrong answer with a confident citation.
- Citation drift when documents change. A citation that pointed at page 4 of version 2 may point at different content in version 3. Citations have to carry the version, not just the page.
- Embedding model version changes invalidating the index. A new embedding model produces vectors in a different space. Old and new vectors are not comparable, so a model change means re-embedding the whole corpus or running two indexes side by side during migration. (Standard practice.)
- Hybrid blend tuning. The weights between vector scores and keyword scores are not set once. They drift as the corpus and the questions change, and the retrieval path log is the only evidence for retuning. (Standard practice.)
- Cost of re-embedding at scale. Re-embedding millions of chunks after a model change or a chunking change is a large batch job with real compute cost. Budget for it before changing either. (Standard practice.)
- Monitoring blind spots. Ingestion success, parsing quality, retrieval recall, and answer faithfulness are four different signals. Watching only one leaves the others free to degrade silently. (Standard practice.)
- Chunking changes invalidate embeddings. Changing the chunk size or the overlap changes every chunk, which means re-embedding everything. Chunking parameters are part of the index contract, not a free tuning knob. (Standard practice.)

12. Open items pinned for later

These were explicitly pinned during the discussion for a follow-up session.

1. Similarity-search algorithms, deeper dive. How HNSW and IVF actually work, their tuning parameters, their tradeoffs, and when to pick one over the other.
2. Common practices. What production retrieval systems do in practice for chunking, embedding, blending, and evaluation, beyond the first design.
3. Best system design. Taking the design in this document plus the algorithm and practice deep

dives, and converging on the recommended architecture.

4. Knowledge base versus knowledge graph. The discussion opened with "not really knowledge graph" and then covered a graph store for entities and relations. The follow-up should settle where the graph store ends and the knowledge base begins, and whether the graph is an index, a product surface, or both.

Appendix: diagrams

Four diagrams accompany this write-up, in `diagrams/` and combined in `diagrams.pdf`:

1. `01-end-to-end-architecture.png`: the full pipeline from the SaaS source to the Q and A layer.
2. `02-ingestion-change-detection.png`: the fetcher loop, cursor table, rate limiting, S3 versioning, and events.
3. `03-query-flow.png`: the hybrid retrieval flow from user query to cited answer.
4. `04-evaluation-loop.png`: evaluation and monitoring at each stage.